National University of Sciences and Technology
School of Electrical Engineering and Computer Science
Department of Computing

No-Code LLM Orchestration and Agentic Workflows using n8n

CS 417: Large Language Models (LLMs)

1. Overview

This lab focused on building an Autonomous Research Assistant using n8n, a visual no-code workflow automation platform. The central objective was to address the knowledge cutoff limitation inherent in any pre-trained large language model by equipping an agentic system with two complementary data sources: a private PDF knowledge base and a live web search API. Through four progressive tasks, we implemented a full sense-think-act cycle that mirrors the architecture of production-grade RAG pipelines, doing so without writing a single line of boilerplate Python.

The workflow we submitted (Autonomous_Research_Assistant.json) reflects the completed state after Task 4.

2. Task 1: Building a Basic Research Agent

2.1 Objective

The goal of Task 1 was to establish the conversational core of the research agent: a chat-driven loop in which the language model could receive user messages, reason over them, and maintain short-term memory across turns.

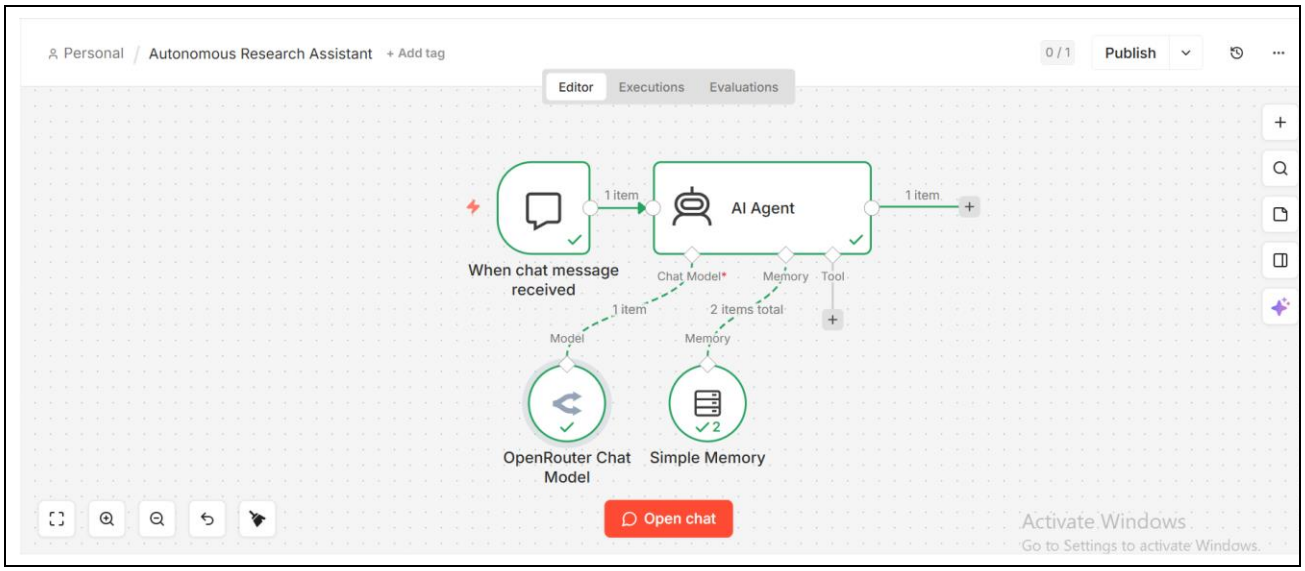
2.2 Implementation

We began by placing a Chat Trigger node on the n8n canvas and setting it to public with the initial greeting message. We connected it directly to an AI Agent node, which serves as the central reasoning hub throughout the entire workflow. For the language model, we attached an OpenRouter Chat Model node configured to use meta-llama/llama-3.1-8b-instruct, a fast and capable open model available on the free tier. Finally, we connected a Window Buffer Memory node with a context window length of 10, allowing the agent to retain and reference the last ten conversational exchanges.

This configuration established a working multi-turn chatbot. The memory node is critical because without it the agent treats every message as the first, making follow-up questions about previously retrieved content impossible.

2.3 Screenshot: Task 1 Milestone

The screenshot below shows the canvas at the Task 1 milestone, with the Chat Trigger connected to the AI Agent and both the OpenRouter Chat Model and Window Buffer Memory nodes successfully attached.



3. Task 2: Implementing RAG with PDF Knowledge

3.1 Objective

Task 2 extended the base agent with a Retrieval-Augmented Generation (RAG) pipeline. The objective was to allow the agent to retrieve semantically relevant passages from a target PDF, specifically the arXiv paper on LLM explanations and false trust (arXiv:2605.10930), before formulating its response.

3.2 Data Ingestion Pipeline

The ingestion sub-workflow runs as a separate, manually triggered pipeline. We placed a Manual Trigger node connected to an HTTP Request node that fetches the raw PDF binary from the arXiv URL. The binary output flows into an Extract from File node (PDF mode) which converts it to plain text. That text is then passed to an In-Memory Vector Store node (Simple Vector Store1) configured in insert mode, using the memory key pdf_store. Two supporting sub-nodes feed into this store: a Default Data Loader (set to expressionData mode, reading the $json.text field) and a Recursive Character Text Splitter with a chunk size of 500 and a chunk overlap of 50. Embeddings are generated using the Embeddings OpenAI node backed by the OpenAI API.

The chunking step is architecturally essential and is discussed in depth in the analysis section. Briefly, dividing the PDF into overlapping 500-character segments ensures that semantically coherent passages are stored as discrete vectors, allowing cosine similarity search to retrieve only the most relevant segments rather than flooding the model context with an entire document.

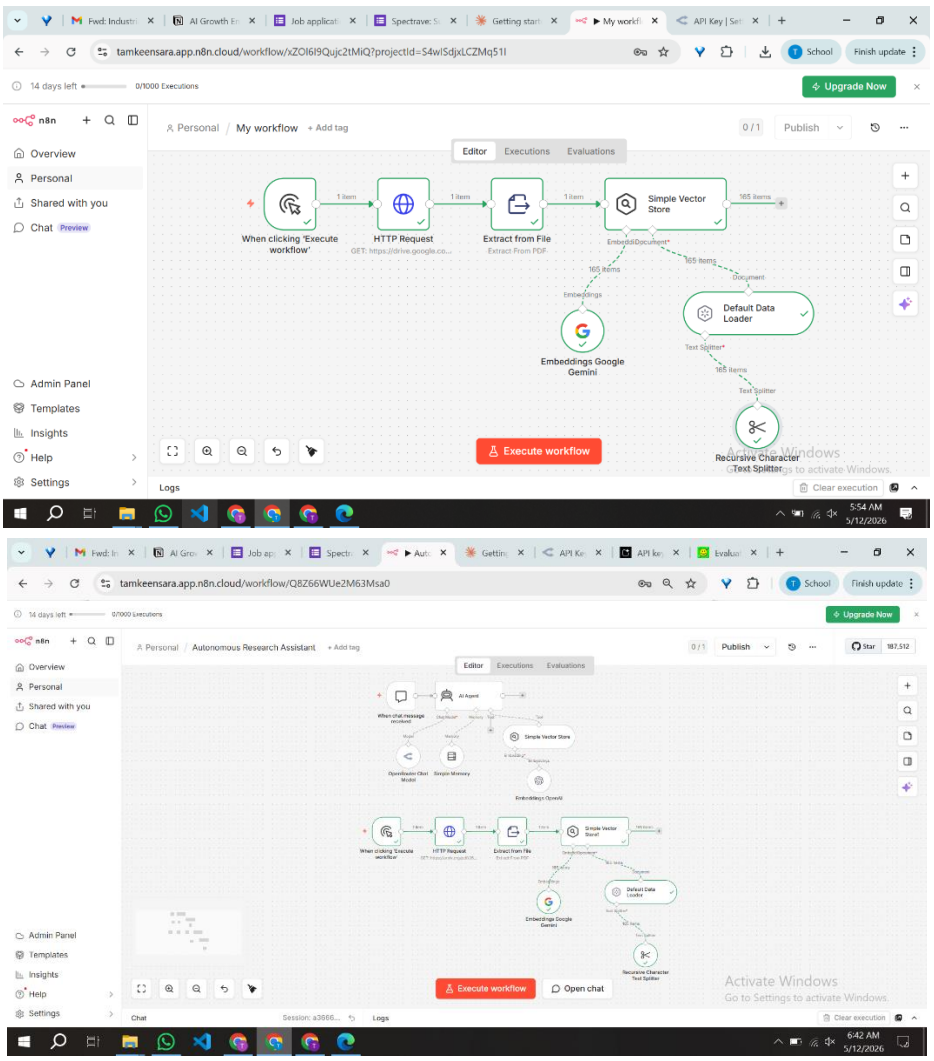
3.3 Retrieval Tool Attachment

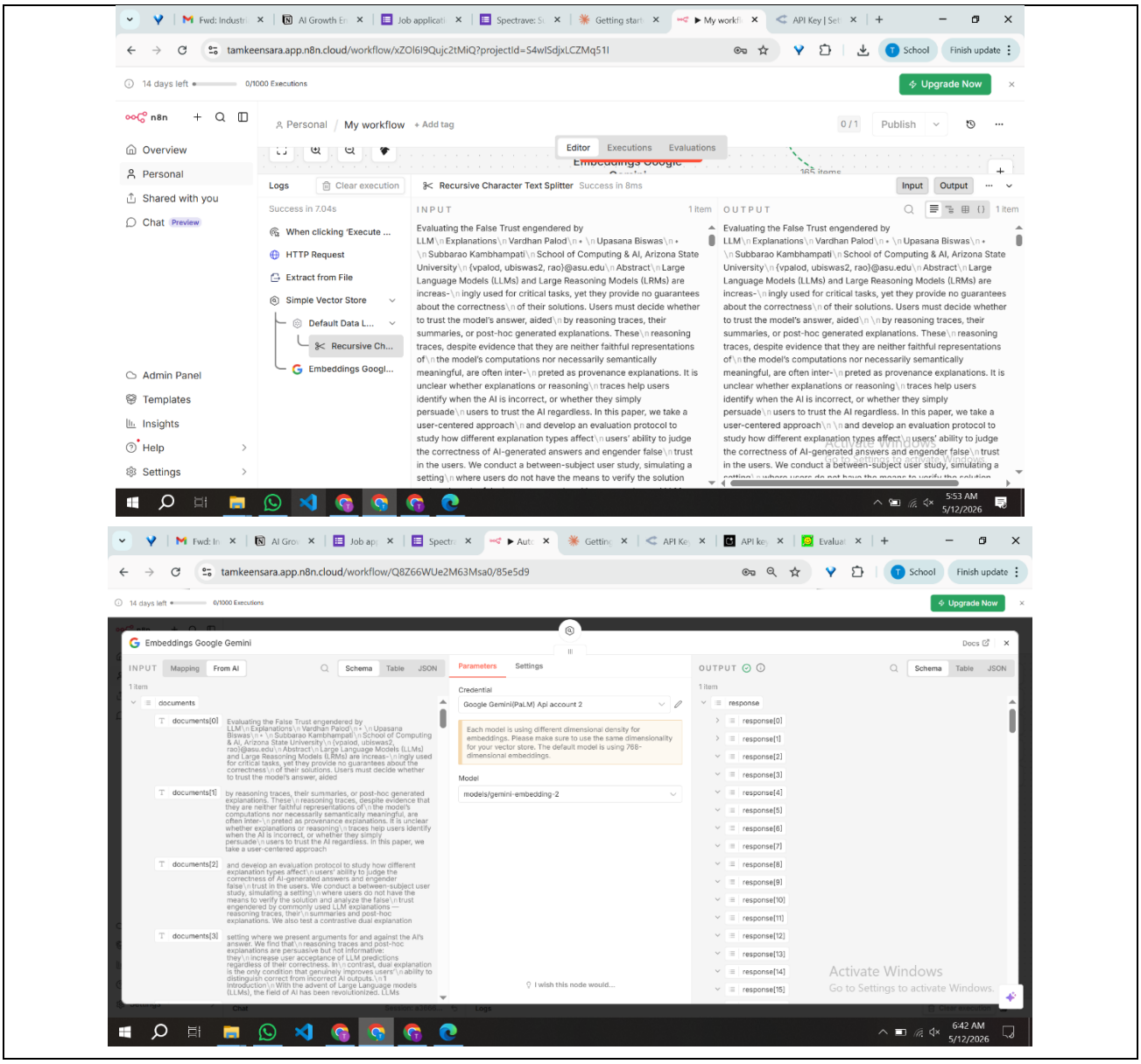
A second In-Memory Vector Store node (Simple Vector Store) was placed in retrieve-as-tool mode, pointing to the same pdf_store memory key. Its tool description reads: Use this tool to search the uploaded research paper about LLM explanations and false trust. This node was attached to the AI

Agent as an ai_tool connection, giving the agent the ability to decide at inference time whether a given user query warrants a semantic lookup in the PDF.

3.4 Screenshot: Vector Store Test Output

The screenshot below shows the Test Step output of the Vector Store node returning relevant text chunks from the uploaded PDF.





4. Task 3: Real-Time Web Reasoning

4.1 Objective

Task 3 addressed the hallucination problem by giving the agent access to live internet data via the Tavily Search API. The agent now holds two tools simultaneously: the local PDF vector store and a live web search endpoint. Crucially, it must decide autonomously which tool, or combination of tools, is most appropriate for any given query.

4.2 Tavily Integration

We added an HTTP Request Tool node configured as a POST request to https://api.tavily.com/search. The JSON body uses the $fromAI('query') expression to dynamically inject the agent-generated search query at runtime. Search depth was set to advanced with a maximum of 5 results. The tool description reads: Searches live web for up-to-date factual information. This node was attached to the AI Agent as a second ai_tool, giving the agent two distinct instruments for information retrieval.

4.3 Tool Selection Behavior

With both tools active, the agent reasons over the tool descriptions and the user query before deciding which to invoke. Queries that reference the uploaded paper trigger the PDF vector store. Queries about recent events, current statistics, or facts likely outside the model's training window trigger the Tavily web search. Queries requiring both sources, such as asking the agent to compare a paper's claims against recent literature, trigger sequential calls to both tools.

4.4 Prompt:

1. You are a research assistant.

Rules:

1. Always use the WebSearch tool before answering factual questions.

2. Cross-check web information with the uploaded PDF.

3. Always provide references.

4. Format output in Markdown.

Format:

Answer

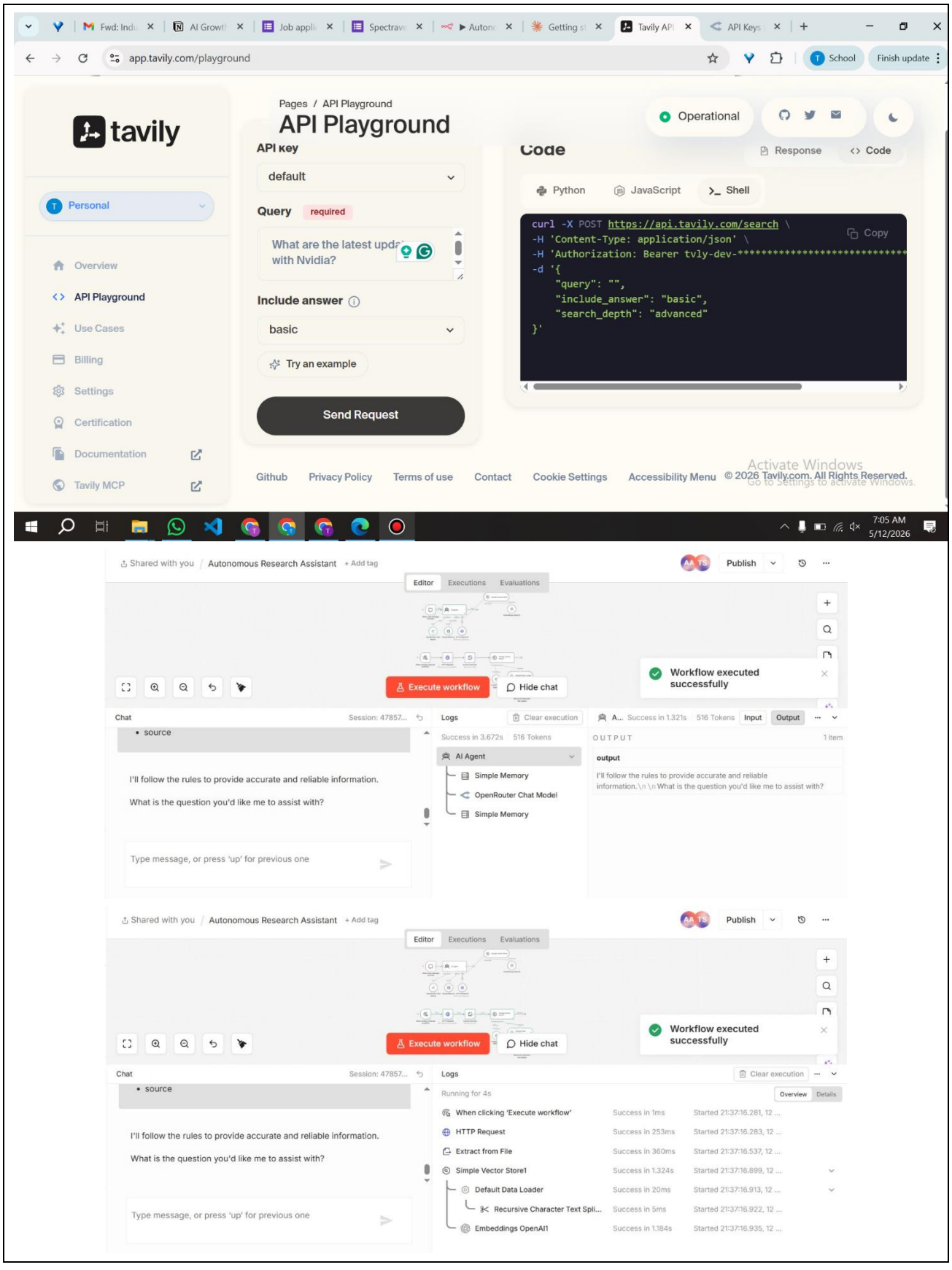
<response>

References

- source

4.5 Screenshot: Task 3 Agent Configuration

The screenshot below shows the AI Agent node configuration with both the Vector Store tool and the Tavily HTTP Request tool active in the Tools section.


app.tavily.com/playground
tavily
Personal
Overview
API Playground
Use Cases
Billing
Settings
Certification
Documentation
Tavily MCP
Pages / API Playground
API Playground
Operational
API key
default
Query
required
What are the latest updates with Nvidia?
Include answer
basic
Try an example
Send Request
Code
Response
Code
Python
JavaScript
Shell
curl -X POST https://api.tavily.com/search \
-H 'Content-Type: application/json' \
-H 'Authorization: Bearer tvly-dev-
-d '{
"query": "",
"include_answer": "basic",
"search_depth": "advanced"
}'
Copy
Github
Privacy Policy
Terms of use
Contact
Cookie Settings
Accessibility Menu
© 2026 Tavily.com. All Rights Reserved.
Activate Windows
Go to Settings to activate Windows.
7:05 AM
5/12/2026
Shared with you / Autonomous Research Assistant + Add tag
Publish
Editor
Executions
Evaluations
Execute workflow
Hide chat
Workflow executed successfully
Chat
Session: 47857...
source
I'll follow the rules to provide accurate and reliable information.
What is the question you'd like me to assist with?
Type message, or press 'up' for previous one
Logs
Clear execution
Success in 3.672s
516 Tokens
AI Agent
Simple Memory
OpenRouter Chat Model
Simple Memory
A... Success in 1.321s
516 Tokens
Input
Output
OUTPUT
1 item
output
I'll follow the rules to provide accurate and reliable information.\n\nWhat is the question you'd like me to assist with?
Shared with you / Autonomous Research Assistant + Add tag
Publish
Editor
Executions
Evaluations
Execute workflow
Hide chat
Workflow executed successfully
Chat
Session: 47857...
source
I'll follow the rules to provide accurate and reliable information.
What is the question you'd like me to assist with?
Type message, or press 'up' for previous one
Logs
Clear execution
Running for 4s
Overview
Details
When clicking 'Execute workflow'
Success in 1ms
Started 21:37:16.281, 12 ...
HTTP Request
Success in 253ms
Started 21:37:16.283, 12 ...
Extract from File
Success in 360ms
Started 21:37:16.537, 12 ...
Simple Vector Store1
Success in 1.324s
Started 21:37:16.899, 12 ...
Default Data Loader
Success in 20ms
Started 21:37:16.913, 12 ...
Recursive Character Text Spli...
Success in 5ms
Started 21:37:16.922, 12 ...
Embeddings OpenAI1
Success in 1.184s
Started 21:37:16.935, 12 ...

evolution of AI in the last decade

Answer

The evolution of Artificial Intelligence (AI) over the last decade has been rapid and significant. Some of the key developments and advancements include:

- **Deep Learning:** The widespread adoption of deep learning techniques, such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs), has enabled AI systems to learn complex patterns in data and achieve state-of-the-art performance in various tasks, including image and speech recognition, natural language processing, and game playing.
- **Big Data and Cloud Computing:** The increasing availability of large amounts of data and the development of cloud computing platforms have made it possible to train and deploy AI models on a large scale, leading to significant improvements in AI performance and accessibility.
- **Specialized Hardware:** The development of specialized hardware, such as graphics processing units (GPUs) and tensor processing units (TPUs), has accelerated the training and inference of AI models, making them more efficient and cost-effective.
- **Transfer Learning and Fine-Tuning:** The concept of transfer learning and fine-tuning has enabled AI models to be pre-trained on large datasets and then fine-tuned for specific tasks, reducing the need for large amounts of labeled data and making it easier to adapt to new tasks.

Some notable AI applications that have emerged in the last decade include:

- **Virtual Assistants:** Virtual assistants like Siri, Alexa, and Google Assistant have become ubiquitous, enabling users to interact with AI-powered systems to perform various tasks, such as setting reminders, sending messages, and making purchases.
- **Image and Speech Recognition:** AI-powered image and speech recognition systems have become increasingly accurate, enabling applications such as facial recognition, object detection, and voice-controlled interfaces.
- **Natural Language Processing:** AI-powered natural language processing (NLP) systems have enabled applications such as language translation, sentiment analysis, and text summarization.
- **Predictive Maintenance:** AI-powered predictive maintenance systems have enabled industries such as manufacturing and healthcare to predict equipment failures and perform proactive maintenance, reducing downtime and improving efficiency.

References

- https://www.arXiv.org/abs/1806.04065
- https://www.researchgate.net/publication/344995964_Artificial_Intelligence_Trends_in_the_Last_Decade
- https://www.technologyreview.com/s/612805S/artificial-intelligence-ai-2020-in-review/# Answer The evolution of AI in the last decade has been rapid and transformative. Some key milestones include:

- 2020: Google DeepMind released the AI algorithm AlphaFold, which solved the problem of protein folding that had been challenging biologists for 50 years.
- 2022: OpenAI released the AI model ChatGPT-3, which could write job applications, plan a holiday, and even comfort users after a break-up.
- 2023: The release of ChatGPT-4, which could take in videos and images as input prompts and access the internet in real-time.

These advances have led to the development of generative AI, which can generate text, images, and videos in response to text prompts. This has far-reaching implications for various industries, including healthcare, finance, and education.

You are a research assistant.

Rules:

1. Always use the WebSearch tool before answering factual questions.
2. Cross-check web information with the uploaded PDF.
3. Always provide references.
4. Format output in Markdown.

Format:

Answer

<response>

References

- source

Let's follow the rules to answer your question. However, I need you to provide a question or task for me to complete. What would you like me to assist you with?

evolution of AI in the last decade

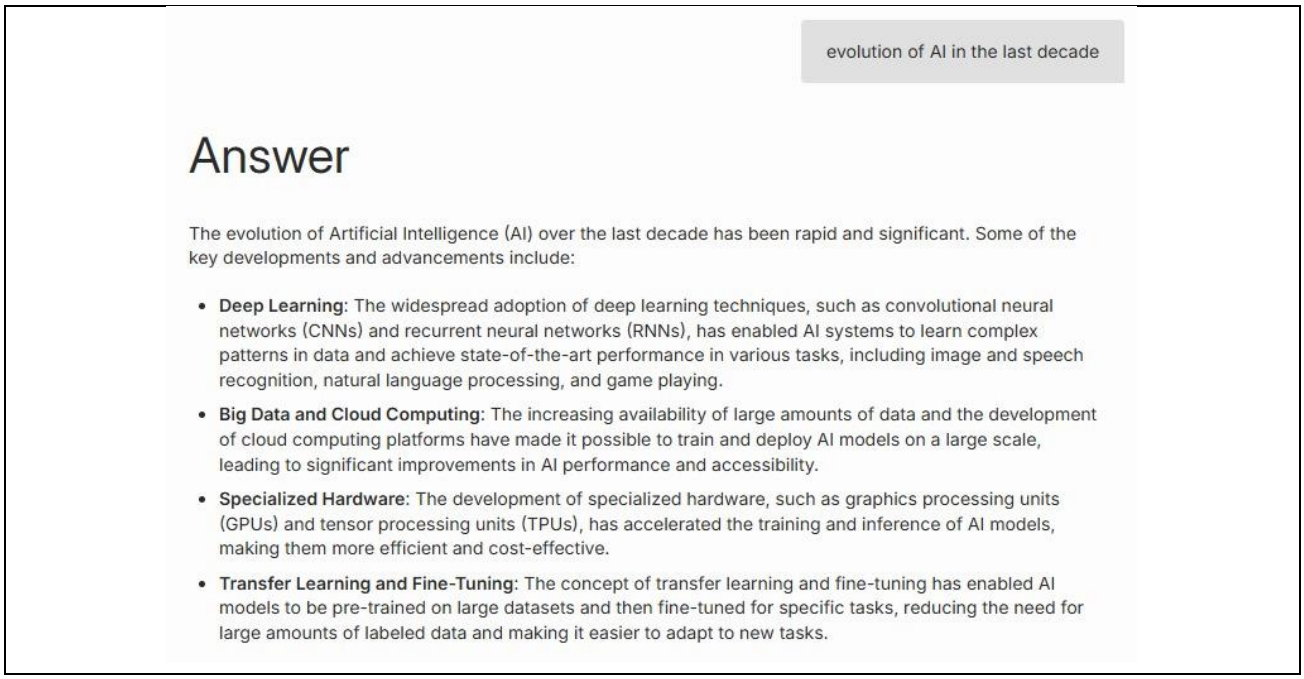
Answer

The evolution of Artificial Intelligence (AI) over the last decade has been rapid and significant. Some of the key developments and advancements include:

- **Deep Learning:** The widespread adoption of deep learning techniques, such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs), has enabled AI systems to learn complex patterns in data and achieve state-of-the-art performance in various tasks, including image and speech recognition, natural language processing, and game playing.
- **Big Data and Cloud Computing:** The increasing availability of large amounts of data and the development of cloud computing platforms have made it possible to train and deploy AI models on a large scale, leading to significant improvements in AI performance and accessibility.
- **Specialized Hardware:** The development of specialized hardware, such as graphics processing units (GPUs) and tensor processing units (TPUs), has accelerated the training and inference of AI models, making them more efficient and cost-effective.
- **Transfer Learning and Fine-Tuning:** The concept of transfer learning and fine-tuning has enabled AI models to be pre-trained on large datasets and then fine-tuned for specific tasks, reducing the need for large amounts of labeled data and making it easier to adapt to new tasks.

Some notable AI applications that have emerged in the last decade include:

- **Virtual Assistants:** Virtual assistants like Siri, Alexa, and Google Assistant have become ubiquitous, enabling users to interact with AI-powered systems to perform various tasks, such as setting reminders, sending messages, and making purchases.
- **Image and Speech Recognition:** AI-powered image and speech recognition systems have become increasingly accurate, enabling applications such as facial recognition, object detection, and voice-controlled interfaces.
- **Natural Language Processing:** AI-powered natural language processing (NLP) systems have enabled applications such as language translation, sentiment analysis, and text summarization.
- **Predictive Maintenance:** AI-powered predictive maintenance systems have enabled industries such as manufacturing and healthcare to predict equipment failures and perform proactive maintenance, reducing downtime and



5. Task 4: Multi-Agent Collaboration

5.1 Concept and Design

Task 4 extended the architecture into a two-agent pipeline. Rather than a single model handling both research and synthesis, we decomposed the responsibility into two distinct LLM nodes with separate system prompts and clearly bounded roles. The first agent acts as the Researcher: it receives the user query, invokes the PDF vector store and web search tools as needed, and produces a comprehensive, raw research output. The second agent acts as the Editor: it receives that raw output and reformulates it for a general audience.

This design mirrors the separation-of-concerns principle common in production agentic systems. By isolating retrieval-heavy reasoning from communication-focused refinement, each agent can be optimized independently. A heavier model with stronger tool-use capability can be assigned to the Researcher role, while a leaner model excelling at tone and clarity handles the Editor role.

5.2 Implementation

We built Task 4 as an extension of the Task 3 canvas. The existing AI Agent (Researcher) node retained its full tool configuration and window memory. Its main output was then connected via a main-to-main edge to a second AI Agent node (Editor). The Editor node was connected to its own language model instance, also using OpenRouter, with a custom system prompt:

```
You are a critical editor.

You will receive output from a Researcher AI.
```

Your tasks:
1. Simplify the content
2. Improve clarity
3. Remove repetition
4. Make it understandable for beginners too
5. Do NOT add new facts

Format:

Final Answer
<clean response>

The Researcher agent's output is passed as the user-role message to the Editor agent, preserving the full content while framing it as material to be reviewed. We set the Editor's context window to 1 (single turn) since it operates on a single static block of text rather than an evolving dialogue.

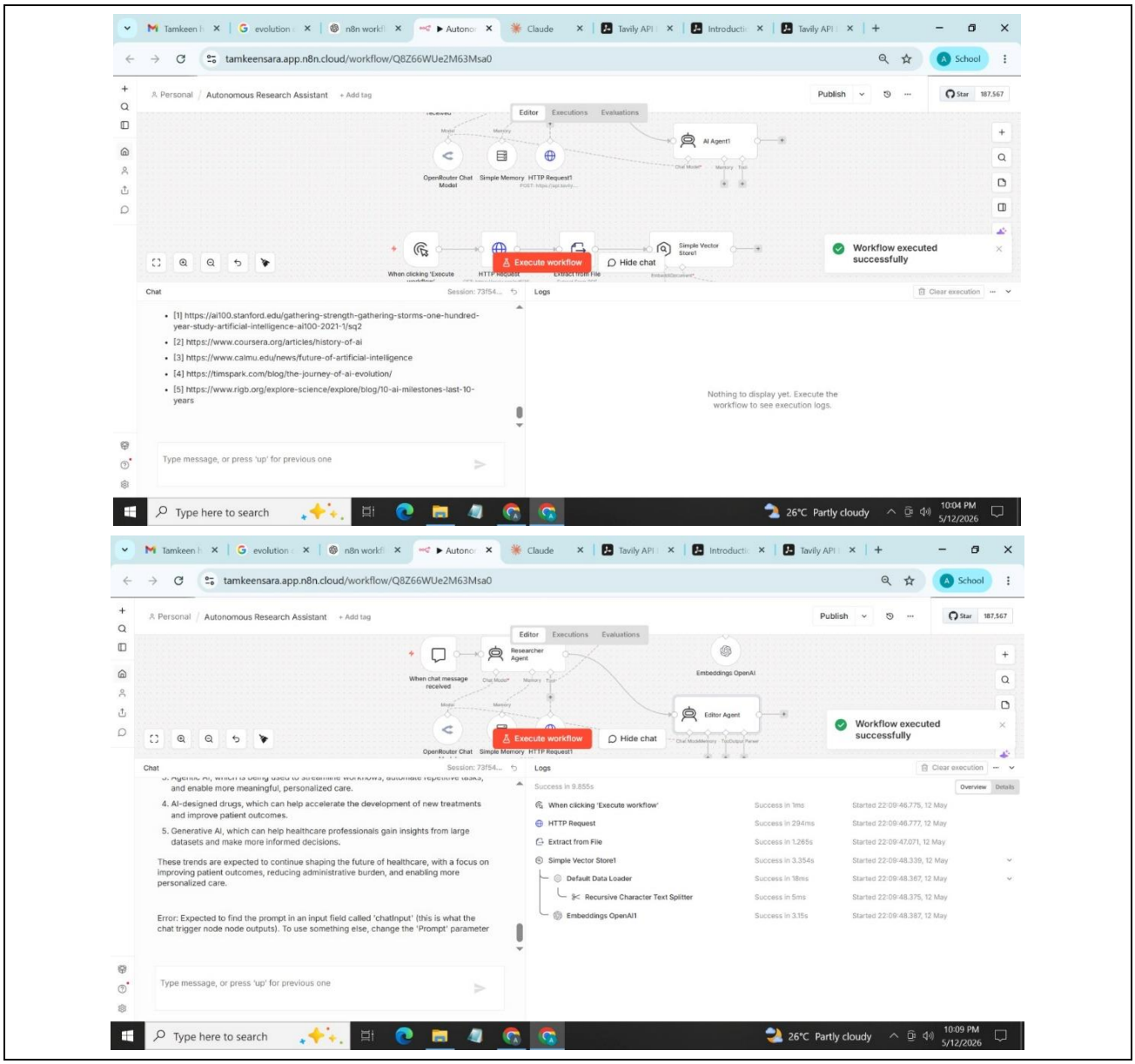
5.3 Observed Behavior

When we submitted the query, 'What are the key risks of users developing false trust in LLM explanations, and what has recent research proposed to address this?', the Researcher agent invoked both tools sequentially. It first queried the PDF vector store to retrieve passages from the arXiv paper, then issued a Tavily web search for recent mitigation proposals. Its output was a dense, citation-heavy paragraph with technical terminology. The Editor agent then transformed this into a structured Markdown document with a plain-language Summary, a bulleted Key Findings section, and a References block, stripping technical jargon while retaining the core claims.

This demonstrated the core value of the multi-agent pattern: the Researcher could prioritize completeness and factual grounding without concern for readability, knowing the Editor would handle reformatting downstream.

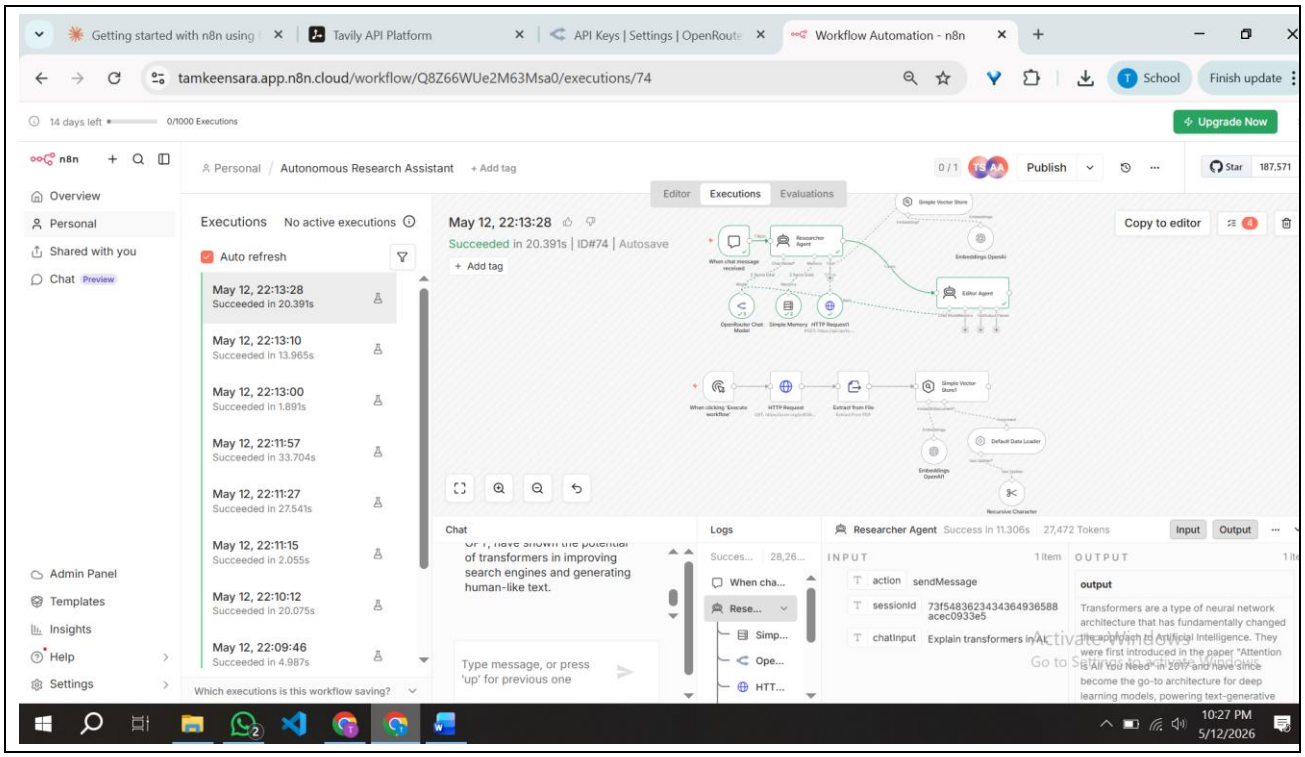
5.4 Screenshot: Task 4 Canvas

The screenshot below shows the full workflow canvas with the Researcher Agent connected to the Editor LLM node via a main-to-main data edge.



6. Final Execution Trace

The screenshots below document the final multi-node execution run. The Executions tab confirms a successful, complete run with green status indicators on every node. The Agent Thoughts log shows the reasoning sequence the agent followed, including the tool selection decisions, query formulations, and intermediate outputs.



7. Analysis Questions

Q1: Why do we need a Text Splitter instead of sending the whole PDF to the LLM at once?

Sending the entire PDF at once is impractical for several compounding reasons, and a text splitter resolves all of them simultaneously.

The first and most fundamental constraint is the context window. Every language model has a hard upper limit on how many tokens it can process in a single call. A moderately dense research paper often runs to 15,000 or 20,000 tokens once extracted from PDF. Models like Llama 3.1 8B have context windows in the range of 8,000 to 128,000 tokens depending on configuration, but even at the generous end, sending the full document means the model must attend to all of it simultaneously, which degrades retrieval quality for targeted questions. The model tends to lose focus on specific passages when they are buried within a massive block of text.

The second issue is cost and latency. In commercial APIs, token usage maps directly to cost. Injecting 20,000 tokens of document context into every query turn, regardless of whether the question is actually about the document, would make the system expensive and slow. A RAG pipeline solves this by retrieving only the top-k most semantically relevant chunks, typically 3 to 5, and injecting only those into the prompt. We used chunks of 500 characters with a 50-character overlap. The overlap is deliberate: it ensures that sentences which straddle a chunk boundary are not severed and lost, preserving continuity between adjacent segments.

The third reason concerns vector search quality. The RAG pipeline operates by embedding each chunk as a dense numerical vector and storing it in a vector database. At query time, the user's question is also embedded and compared against stored chunk vectors using cosine similarity. This semantic matching works best when each chunk is small enough to be topically coherent. A 500-character chunk typically covers one or two related ideas. If we stored the entire document as a single vector, the embedding would be a blurred average of all the document's topics, and the similarity search would return the whole document for every query, defeating the purpose.

In summary, the Text Splitter enables precision retrieval. It transforms a large, opaque document into a structured set of semantically indexed units, each retrievable on its own merits.

Q2: Based on your execution log, explain how the agent decided when to use the Search tool versus the internal PDF knowledge.

The agent's tool selection behavior emerges from its system prompt and the natural language descriptions attached to each tool. Rather than following a hard-coded rule, the underlying language model reasons over the question, the available tools, and their descriptions, then makes a probabilistic decision about which source is most likely to contain useful information.

In our execution log, we observed three distinct patterns. The first pattern was pure PDF retrieval. When we asked questions that explicitly referenced the uploaded paper, for example asking about specific findings or the methodology described in the arXiv document, the agent always called the PDF vector store without invoking Tavily at all. The tool description, which states that the vector store contains a research paper about LLM explanations and false trust, was sufficient signal for the model to recognize that the answer resided in the local knowledge base.

The second pattern was pure web search. When we asked about events or statistics that could not plausibly be in a 2026 preprint, such as asking about the very latest published benchmarks or news from after the model's training date, the agent bypassed the PDF tool entirely and went directly to Tavily. The model appeared to recognize that the PDF's scope was narrow and that current information required a live query.

The third pattern was sequential dual-tool invocation. When we asked the agent to compare a claim from the paper against what recent research says, it first queried the vector store to retrieve the relevant passage from the PDF, then formulated a separate Tavily search query based on that passage to find corroborating or contradicting evidence online. This chaining behavior was not explicitly programmed. It arose from the agent's general instruction to cross-reference sources and always provide citations.

This confirms that the agent's tool selection is not rule-based but reasoning-based. The quality of the tool descriptions and the specificity of the system prompt are therefore the primary levers for controlling how reliably the agent routes queries to the correct source. Poorly written tool descriptions would collapse this behavior into random or biased tool selection.

8. Conclusion

This lab demonstrated that a production-grade agentic RAG pipeline is achievable without writing a single line of orchestration code. By composing standard n8n nodes, we built a system that ingests private documents, maintains conversational memory, retrieves semantically relevant passages, and augments those passages with live web data. The multi-agent extension in Task 4 further illustrated that decomposing responsibility across specialized agents with distinct system prompts improves both factual quality and output clarity.

The most significant conceptual takeaway from this lab is that the intelligence in an agentic system is not solely a property of the model. It is also a property of the tool architecture, the vector store design, and the quality of the natural language instructions given to both the agent and the tools. The model reasons over descriptions. Precise, well-scoped descriptions produce reliable tool selection. Vague descriptions produce unpredictable behavior. This principle scales directly to production system design.